

Conjugate Gradient Poisson Solver

Jie-Cheng Lin, Wen-Hung Luo, Ming-Chun Lin, I-Lun Tsai

Department of Physics
National Taiwan University

Jun 2025

1 Introduction

- Introduction

2 Principle

- Principle1
- Principle2
- Principle3
- Principle4

3 Implementation

- Implementation (CG method)
- CG Algorithm Parallelization with OpenMP
- CG Algorithm Parallelization with MPI
- Schematics of Parallelization

- Result of OpenMP
- Schematics of Parallelization
- Schematics of Parallelization

4 Performance

- Performance with Difference BC
- Performance with Difference Grid Size N
- Performance with and w/o Matrix-Free
- Wall-Clock Time vs. Number of cells
- Performance (other plots)
- Performance (other plots)
- Performance (other plots)
- Performance (other plots)

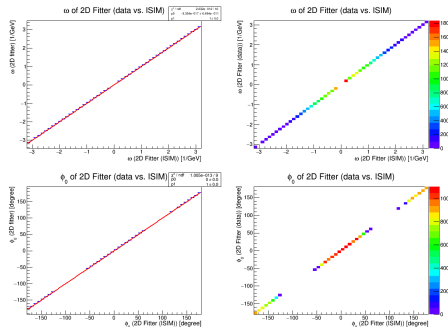
5 Q&A

Introduction

- Implement the Conjugate Gradient (CG) Method with parallel processing to Solve the 2D Poisson Equation.
 - The typical form of the equation is: $\nabla^2 \phi = f(x, y)$.
 - After discretization, it becomes a sparse linear system: $\mathbf{Ax} = \mathbf{b}$.
 - The CG method efficiently solves this system when $\mathbf{A}$ is symmetric and positive-definite.
- Performance Evaluation.
 - Measure execution time at different grid resolutions (number of cells), and analyze the scaling behavior (wall-clock time vs. grid size).
 - Compare the performance with the traditional SOR (Successive Over-Relaxation) method, including number of iterations to converge and execution time.

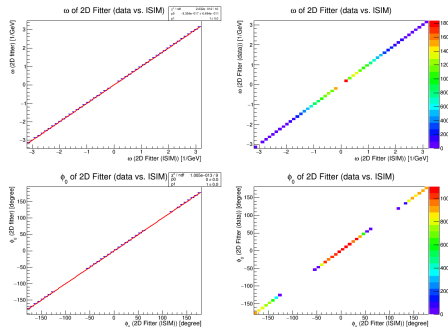
Principle1

- Imagine you walk into a sushi bar. There are many delicious sushi on it. However...



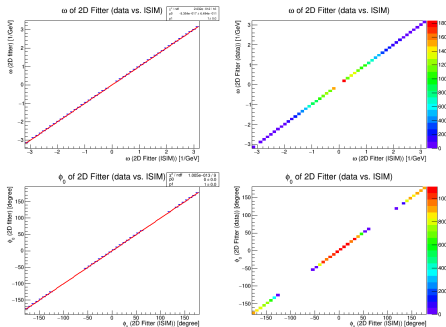
Principle2

- Imagine you walk into a sushi bar. There are many delicious sushi on it. However...



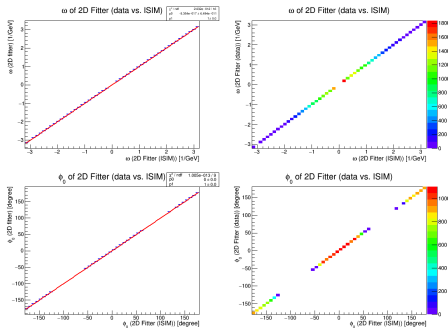
Principle3

- Imagine you walk into a sushi bar. There are many delicious sushi on it. However...



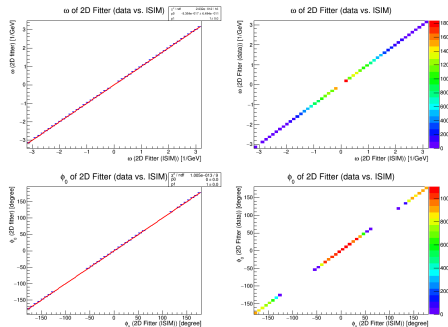
Principle4

- Imagine you walk into a sushi bar. There are many delicious sushi on it. However...



Implementation (CG method)

- Imagine you walk into a sushi bar. There are many delicious sushi on it. However...



- Parallelizable Operations

- Shared Memory Environment: All threads access shared data structures (matrix and vectors).
- Parallel Matrix-Vector Multiplication: Distribute loop iterations evenly across threads.
- Parallel Reduction for Inner Products: Sum computed in parallel using OpenMP reduction clauses.
- Synchronization: OpenMP implicitly synchronizes threads at the end of parallel regions.

CG Algorithm Parallelization with MPI

- MPI Algorithm Steps
 - Distribute Matrix and Vectors among MPI processes.
 - Local Computation: Each MPI process computes local parts of matrix-vector products and inner products.
 - Global Reduction: MPI_All reduce synchronizes and sums local computations for global values (e.g., residual norm, dot products).
 - Iteration Update: Update vectors x, r, p_x, r, p independently after global synchronization.

Schematics of Parallelization

Initial Guess (x_0), distribute data among MPI processes



Compute $r_0 = b - Ax_0$ (parallel matrix-vector product)



Set $p_0 = r_0$



For $k = 0, 1, 2, \dots$ (Until convergence):

OpenMP (within each MPI process):

- Parallel loops for vector operations
- Parallel reduction for inner products

MPI:

- Local ($A p_k$), local dot products
- Global MPI_Allreduce (combine data)



Converged solution x_n

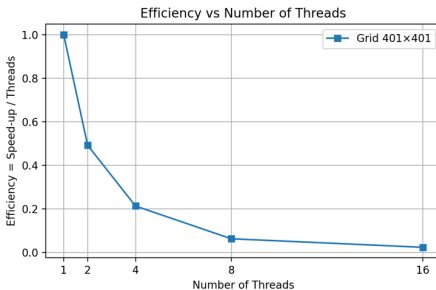
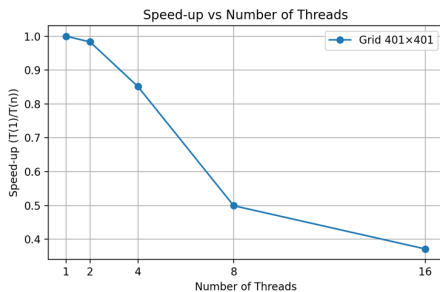
Update:

$\alpha_k, x_{k+1}, r_{k+1}, \beta_k, p_{k+1}$

Check convergence criterion

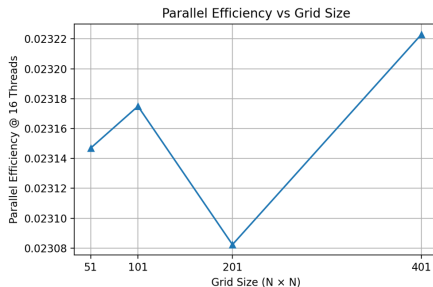
Result of OpenMP

- False sharing / cache contention
- Insufficient computational workload per thread
- Overhead of thread management
- Critical sections or reduction barriers.
- No data locality or cache blocking
- Too much parallel overhead for this problem size



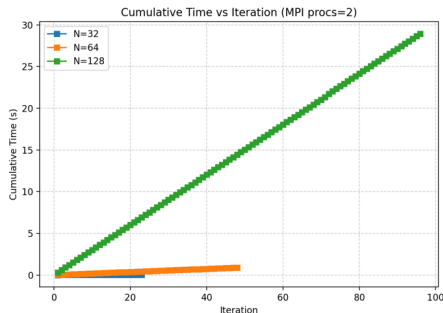
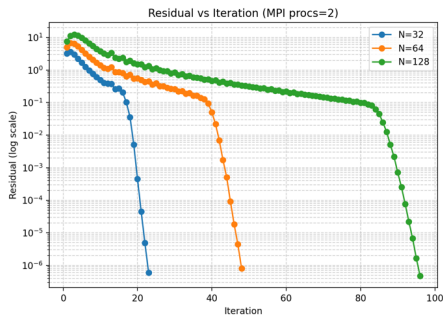
Schematics of Parallelization

- Low Efficiency at Grid Size 201×201
 - Load Imbalance
 - Cache and Memory Issues
 - Synchronization Overhead



Schematics of Parallelization

- A Larger grid means more unknowns and a more complex linear system.
- CG convergence is affected by the condition number of matrix A, which increases with grid size, slowing convergence.
- More data to distribute and synchronize across MPI processes.
- Larger N implies matrix-vector products scale as $O(N^2)$ to $O(N^3)$ depending on implementation.



Performance with Difference BC

- When the grid size N is **small**, using **boundary condition 1** results in a larger number of CG iterations. This is due to oscillations caused by the **asymmetry** of the matrix associated with BC1.

```
caiyilun@caiyilunde-MacBook-Pro code % ./final_project 1 false 10 10
設定後 Nx : 10, Ny : 10
mkdir: RESULT: File exists
```

```
-----
Nx : 10, Ny : 10
Tolerance : 1.000000e-08, Omega : 1.800000
```

```
[CG Method]
CG_1 method - Error : 6.433132e-04, Iteration : 2563, Time : 0.001827 s
```

```
[SOR Method]
SOR_1 Method - Error : 3.469912e-04, Iteration : 61, Time : 0.000132 s
caiyilun@caiyilunde-MacBook-Pro code %
```

```
if (argc >= 2) BC = atoi(argv[1]);
if (argc >= 3) use_matrix_A = (strcmp(argv[2], "true") == 0);
if (argc >= 4) ROW = atoi(argv[3]);
if (argc >= 5) COL = atoi(argv[4]);
```

Difference BC

```
caiyilun@caiyilunde-MacBook-Pro code % ./final_project 2 false 10 10
設定後 Nx : 10, Ny : 10
mkdir: RESULT: File exists
```

```
-----
Nx : 10, Ny : 10
Tolerance : 1.000000e-08, Omega : 1.800000
```

```
[CG Method]
CG_2 method - Error : 7.908507e-06, Iteration : 7, Time : 0.000040 s
```

```
[SOR Method]
SOR_2 Method - Error : 7.908508e-06, Iteration : 85, Time : 0.000333 s
caiyilun@caiyilunde-MacBook-Pro code %
```

Performance with Difference Grid Size N

- As the number of grid points **increases**, the number of iterations rises significantly. At the same time, the influence of boundary conditions on the Conjugate Gradient (CG) method gradually diminishes, the **accuracy of the numerical solution improves**, but this also comes with a **longer computation time**.

```
-----
Nx : 50, Ny : 50
Tolerance : 1.000000e-08, Omega : 1.800000
-----

[CG Method]
CG_1 method - Error : 2.257441e-05, Iteration : 170, Time : 0.004157 s

[SOR Method]
SOR_1 Method - Error : 1.095029e-05, Iteration : 674, Time : 0.042238 s
caiyilun@caiyilunde-MacBook-Pro code % █

-----
Nx : 100, Ny : 100
Tolerance : 1.000000e-08, Omega : 1.800000
-----

[CG Method]
CG_1 method - Error : 5.418124e-06, Iteration : 222, Time : 0.023904 s

[SOR Method]
SOR_1 Method - Error : 2.664048e-06, Iteration : 2193, Time : 0.596733 s
caiyilun@caiyilunde-MacBook-Pro code % █

-----
Nx : 200, Ny : 200
Tolerance : 1.000000e-08, Omega : 1.800000
-----

[CG Method]
CG_1 method - Error : 1.325926e-06, Iteration : 441, Time : 0.175544 s

[SOR Method]
SOR_1 Method - Error : 6.571365e-07, Iteration : 6800, Time : 7.152766 s
caiyilun@caiyilunde-MacBook-Pro code % █
```


Performance with and w/o Matrix-Free

- Enabling the matrix-free mode eliminates the cost of explicitly constructing the sparse matrix, which **significantly improves computational performance and reduces memory usage**, especially for large-scale grids. Compared to the CG method with an explicit matrix, the matrix-free version achieves noticeably **faster runtime**.
- In contrast, the SOR method is unaffected by this setting since it does not rely on a global matrix structure. Overall, the matrix-free approach is particularly well-suited for solving large-scale problems using the CG method.

```
-----  
Nx : 100, Ny : 100  
Tolerance : 1.000000e-08, Omega : 1.800000  
-----
```

With Matrix-Free

```
[CG Method]  
CG_1 method - Error : 5.418124e-06, Iteration : 222, Time : 0.020315 s  
  
[SOR Method]  
SOR_1 Method - Error : 2.664048e-06, Iteration : 2193, Time : 0.554946 s  
caiyilun@caiyilunde-MacBook-Pro code % █
```

```
-----  
Nx : 100, Ny : 100  
Tolerance : 1.000000e-08, Omega : 1.800000  
-----
```

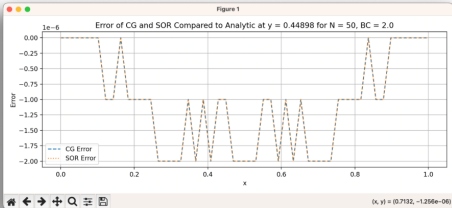
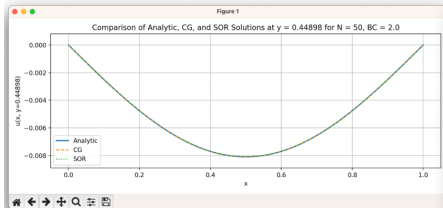
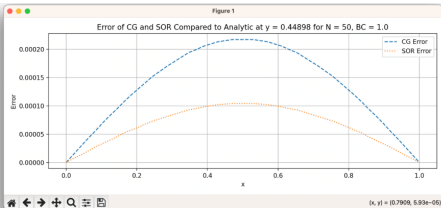
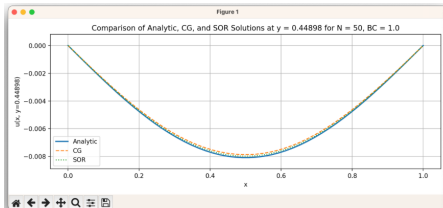
Without Matrix-Free

```
[CG Method]  
CG_1 method - Error : 5.418124e-06, Iteration : 222, Time : 26.813761 s  
  
[SOR Method]  
SOR_1 Method - Error : 2.664048e-06, Iteration : 2193, Time : 0.531892 s  
caiyilun@caiyilunde-MacBook-Pro code % █
```

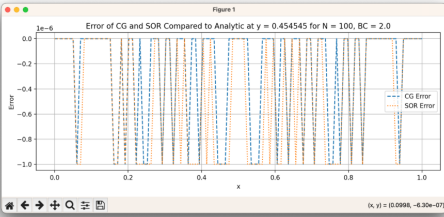
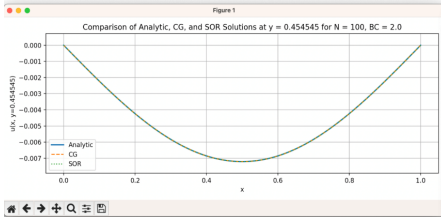
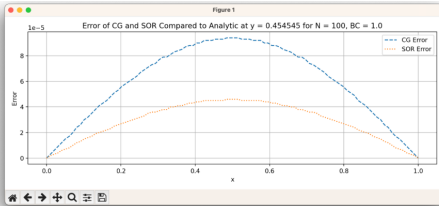
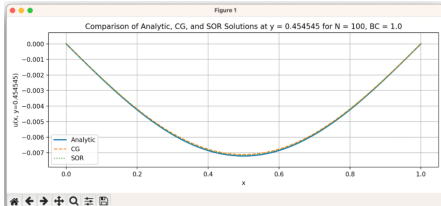
Wall-Clock Time vs. Number of cells

	SOR		CG (with matrix-free)		CG (without matrix-free)	
	Error	Time	Error	Time	Error	Time
N = 50	1.0950e-05	0.041171 s	2.2574e-05	0.004034 s	2.2574e-05	1.162133 s
N = 100	2.6641e-06	0.554324 s	5.4181e-06	0.023376 s	5.4181e-06	27.224622 s
N = 200	6.5714e-07	6.953978 s	1.3259e-06	0.177531 s	none	none

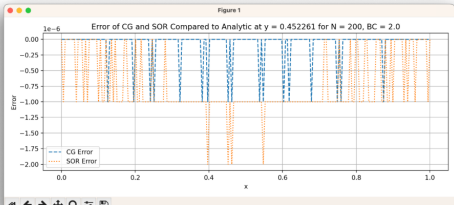
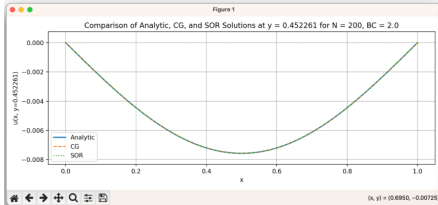
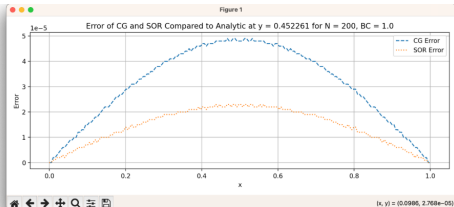
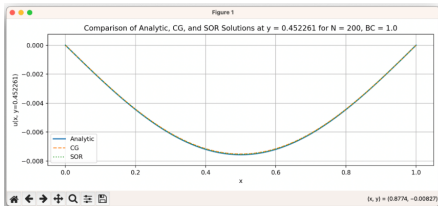
Performance (other plots)



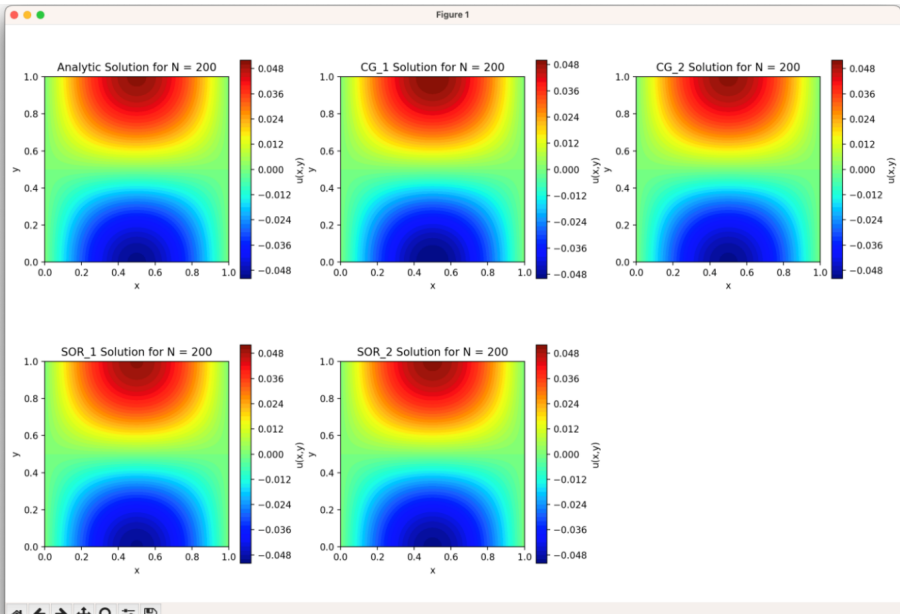
Performance (other plots)



Performance (other plots)



Performance (other plots)



- Q&A